

UNIVERSIDAD PÚBLICA DE EL ALTO

Ingeniería de Sistemas

Aula Virtual Académica

Mini Moodle — Sistema de Gestión de Aprendizaje

ASIGNATURA	TEM-742 Tecnologías Emergentes II
GRUPO	Grupo N° 21
INTEGRANTE	Javier Jose Choque Quispe
DOCENTE	M. Sc. Mario Tórrez C.
FECHA	26 de junio de 2026

Proyecto Final de Asignatura · 2026

Tabla de Contenidos

1.	Planteamiento del Problema y Justificación
2.	Objetivo General y Objetivos Específicos
3.	Descripción de los Módulos Principales
4.	Modelo de Base de Datos
5.	Tecnologías Utilizadas
6.	Uso de Herramientas de Inteligencia Artificial
7.	Conclusiones y Aprendizajes

1. Planteamiento del Problema y Justificación

1.1 Planteamiento del Problema

En la actualidad, las instituciones educativas de nivel secundario y superior enfrentan una creciente demanda de herramientas digitales que faciliten la gestión académica de manera eficiente, centralizada y accesible. Sin embargo, muchos colegios y unidades educativas del entorno boliviano siguen operando con métodos tradicionales basados en papel, comunicación verbal o plataformas no especializadas como grupos de WhatsApp o correo electrónico para la distribución de materiales y tareas.

Esta situación genera múltiples problemas: falta de trazabilidad en la entrega de trabajos, dificultad para que los docentes gestionen calificaciones de forma ordenada, ausencia de un canal oficial para anuncios, y nula visibilidad del progreso académico del estudiante en tiempo real. Además, los docentes no cuentan con herramientas para organizar el contenido de sus cursos en módulos temáticos, lo que dificulta la planificación pedagógica.

1.2 Justificación

El desarrollo del **Aula Virtual Académica** surge como respuesta directa a esta problemática. La plataforma proporciona un entorno web centralizado donde docentes pueden crear cursos organizados por temas, publicar materiales y tareas, gestionar calificaciones con retroalimentación, y comunicarse con sus estudiantes mediante anuncios y notificaciones en tiempo real.

Los estudiantes, por su parte, pueden unirse a cursos mediante un código de acceso único, consultar sus materiales, entregar tareas y visualizar sus calificaciones desde cualquier dispositivo con acceso a internet. El sistema también incorpora un panel administrativo completo que permite al administrador supervisar y moderar toda la actividad de la plataforma.

Desde el punto de vista tecnológico, el proyecto representa una oportunidad para aplicar conceptos de desarrollo web moderno con Python/Flask, arquitectura de aplicaciones en capas, diseño de bases de datos relacionales normalizadas, y construcción de interfaces de usuario responsivas con Tailwind CSS — competencias directamente alineadas con los contenidos de la asignatura TEM-742 Tecnologías Emergentes II.

2. Objetivo General y Objetivos Específicos

2.1 Objetivo General

Desarrollar una aplicación web funcional de gestión de aprendizaje (LMS) llamada **Aula Virtual Académica**, orientada a colegios, que permita a docentes crear y administrar cursos con materiales, tareas y calificaciones, y a estudiantes inscribirse mediante código de acceso, entregar trabajos y consultar su progreso académico, empleando Flask, SQLAlchemy, SQLite y Tailwind CSS bajo el patrón Application Factory.

2.2 Objetivos Específicos

- Diseñar e implementar un modelo de base de datos relacional normalizado con 10 tablas que soporte la jerarquía: cursos → temas → materiales/tareas → entregas → calificaciones.
- Implementar un sistema de autenticación con Flask-Login que soporte tres roles de usuario (administrador, docente, estudiante) con acceso diferenciado a las funcionalidades del sistema.
- Desarrollar el módulo de cursos con generación automática de códigos de acceso de 6 caracteres para que los estudiantes puedan inscribirse de forma autónoma.
- Construir el módulo de tareas con soporte para fecha límite en zona horaria Bolivia (UTC-4), detección automática de entregas tardías y sistema de calificación con retroalimentación.
- Implementar un sistema de notificaciones en tiempo real que informe a los estudiantes sobre nuevas tareas publicadas, anuncios y calificaciones recibidas.
- Desarrollar un panel de administración completo con CRUD de usuarios y cursos, gestión de inscripciones, moderación de anuncios y reporte general de entregas.
- Diseñar una interfaz de usuario profesional y responsiva utilizando Tailwind CSS con soporte para modo oscuro/claro, sidebar colapsable y paleta de colores coherente con estética moderna.
- Desplegar la aplicación en un servicio de nube pública (Render o Heroku) garantizando su accesibilidad desde cualquier dispositivo con conexión a internet.

3. Descripción de los Módulos Principales

El sistema está organizado siguiendo el patrón **Application Factory** con Flask Blueprints. Cada módulo funcional tiene su propio directorio con modelos, rutas y templates independientes, lo que garantiza bajo acoplamiento y alta cohesión. A continuación se describe cada módulo principal.

3.1 Módulo de Autenticación (auth/)

Gestiona el ciclo de vida de la sesión del usuario: registro, inicio de sesión y cierre de sesión. Utiliza **Flask-Login** para mantener la sesión activa y **Werkzeug** para el hashing seguro de contraseñas mediante PBKDF2-SHA256. El sistema soporta tres roles: **admin**, **docente** y **estudiante**, definidos como columna rol en la tabla usuarios.

El decorador `@rol_requerido()` implementado en `app/decorators.py` protege cada ruta verificando que el usuario autenticado tenga el rol necesario antes de procesar la solicitud, devolviendo un error HTTP 403 en caso contrario.

Ruta	Método	Descripción
/auth/login	GET/POST	Formulario de inicio de sesión con validación de credenciales
/auth/register	GET/POST	Registro de nuevos usuarios (solo durante configuración inicial)
/auth/logout	GET	Cierre de sesión y limpieza de cookie de sesión

Tabla 1: Rutas del módulo de autenticación

3.2 Módulo de Usuarios (usuario/)

Centraliza la gestión de cuentas de usuario. El administrador dispone de un CRUD completo: puede listar todos los usuarios con filtros por rol, crear nuevos usuarios con cualquier rol, editar datos personales y contraseña, activar/desactivar cuentas, y eliminar usuarios del sistema. El modelo Usuario expone la propiedad calculada `nombre_completo` y el método `set_password()` que aplica el hash antes de persistir.

Para el acceso del administrador, todas las vistas de gestión de usuarios extienden de `admin/base_admin.html`, que provee el sidebar lateral exclusivo del panel administrativo en lugar del navbar estándar de docentes y estudiantes.

3.3 Módulo de Cursos (curso/)

Es el núcleo del sistema. Los docentes pueden crear cursos personalizando nombre, descripción, sección y color de tarjeta. Al crearse, el sistema genera automáticamente un **código de acceso único de 6 caracteres** (letras mayúsculas + dígitos) mediante un bucle que garantiza unicidad consultando la base de datos.

El flujo de inscripción con código es el siguiente:

Paso	Actor	Acción
1	Docente	Crea el curso → sistema genera código (ej: X7K2PQ)
2	Docente	Comparte el código con sus estudiantes por cualquier medio

Paso	Actor	Acción
3	Estudiante	Va a 'Unirse a curso' e ingresa el código de 6 caracteres
4	Sistema	Verifica el código, valida que el curso esté activo y no haya inscripción previa
5	Sistema	Registra la inscripción — el curso aparece en el dashboard del estudiante

Tabla 2: Flujo de inscripción con código de acceso

3.4 Módulo de Temas y Materiales (tema/ y material/)

Los temas funcionan como módulos dentro del curso, ordenados numéricamente. Cada tema puede contener materiales de tres tipos: **PDF** (enlace o archivo), **video** (carga directa al servidor con reproducción en navegador mediante etiqueta <video>), y **enlace** externo. Los materiales tienen un campo orden que controla su presentación dentro del tema.

3.5 Módulo de Tareas, Entregas y Calificaciones (tarea/)

El módulo de tareas es el más complejo del sistema. Los docentes pueden publicar tareas por tema especificando título, instrucciones, fecha límite con hora exacta y puntaje máximo. El sistema detecta automáticamente si una entrega es **tardía** comparando la fecha de entrega con la fecha límite, ambas en zona horaria Bolivia (UTC-4).

La vista de entregas para el docente muestra un listado filtrable con tres categorías: **Puntuales** (badge verde), **Tardías** (badge rojo con el tiempo de retraso exacto en minutos, horas o días), y todas las entregas combinadas. Al calificar una entrega individual, el sistema también muestra el badge de puntualidad para que el docente pueda considerarlo en la nota asignada.

Una vez calificada una entrega, se genera automáticamente una **notificación** para el estudiante con la nota obtenida. El docente puede reabrir una entrega calificada si el estudiante necesita corregirla.

3.6 Módulo de Anuncios (anuncio/)

Los docentes pueden publicar anuncios en sus cursos. Al publicarse, el sistema envía automáticamente notificaciones a todos los estudiantes inscritos en ese curso. El administrador puede visualizar y eliminar cualquier anuncio del sistema desde su panel dedicado.

3.7 Módulo de Notificaciones (notificacion/)

Sistema de alertas internas que notifica a los usuarios sobre eventos relevantes. Soporta tres tipos de notificaciones: **tarea** (nueva tarea publicada), **calificacion** (entrega calificada con nota) y **anuncio** (nuevo anuncio en curso). El contador de notificaciones no leídas aparece como badge en el ícono de campana del navbar. El usuario puede marcar notificaciones como leídas individualmente.

3.8 Panel de Administración

El administrador tiene un entorno completamente separado de docentes y estudiantes. En lugar del navbar horizontal estándar, dispone de un **sidebar lateral fijo y colapsable** con secciones bien delimitadas. Todas las vistas del panel extienden de admin/base_admin.html, garantizando consistencia visual. Las acciones destructivas (eliminar, desactivar) utilizan el componente components/modal.html en lugar de window.confirm(), proporcionando una experiencia de usuario más cuidada y coherente con el

diseño del sistema.

Sección	Operaciones disponibles
Dashboard	KPIs del sistema: total usuarios, docentes, estudiantes, cursos, inscripciones, entregas pendientes
Usuarios	Listar con filtro por rol, crear, editar datos/contraseña/rol, activar/desactivar, eliminar
Cursos	Ver todos, gestionar (editar nombre/descripción/color/sección, activar/desactivar, eliminar)
Inscripciones	Lista global de todas las inscripciones con buscador, dar de baja a estudiante de un curso
Anuncios	Lista global de anuncios con buscador por título/curso, eliminar anuncios inapropiados
Entregas	Reporte de solo lectura: KPIs + tabla filtrable por estado (calificada/pendiente/tardía)

Tabla 3: Módulos del panel de administración

4. Modelo de Base de Datos

El sistema utiliza un modelo relacional compuesto por **10 tablas normalizadas**, diseñadas en Tercera Forma Normal (3FN). Todas las relaciones están implementadas con claves foráneas y restricciones de integridad referencial gestionadas por SQLAlchemy. Las eliminaciones en cascada garantizan la consistencia de los datos cuando se elimina un curso o usuario.

4.1 Descripción de Tablas

usuarios	Almacena todos los usuarios del sistema. El campo rol ('admin', 'docente', 'estudiante') determina los permisos. El campo activo permite deshabilitar cuentas sin eliminarlas.
cursos	Representa un curso creado por un docente. Contiene el codigo_acceso único de 6 caracteres para inscripción, el campo color para personalización de tarjeta, y activo para control de visibilidad.
inscripciones	Tabla de unión entre usuarios (estudiantes) y cursos. Tiene restricción UNIQUE(estudiante_id, curso_id) para evitar inscripciones duplicadas.
temas	Módulos o unidades temáticas dentro de un curso. El campo orden controla la secuencia de presentación. Los temas pueden ocultarse a los estudiantes.
materiales	Recursos educativos asociados a un tema: PDF, video o enlace externo. El campo tipo determina cómo se renderiza en la vista del estudiante.
tareas	Trabajos asignados dentro de un tema con fecha_limite , puntaje_max y control de visibilidad. La lógica de detección de tardanza se calcula en tiempo de consulta.
entregas	Registro de cada envío de un estudiante para una tarea. El campo estado puede ser 'entregado' o 'calificado'. Permite adjuntar archivo PDF y/o comentario de texto.
calificaciones	Nota y retroalimentación asignadas por el docente a una entrega específica. Relación 1:1 con entregas — una entrega solo puede tener una calificación.
anuncios	Publicaciones del docente en su curso. Al crearse, dispara la generación de notificaciones para todos los estudiantes inscritos.
notificaciones	Alertas internas por usuario. El campo tipo categoriza el evento ('tarea', 'calificacion', 'anuncio') y leido controla el contador del badge.

4.2 Diagrama Relacional (Esquema Textual)

A continuación se presenta el esquema relacional indicando las claves primarias (PK), claves foráneas (FK) y relaciones de cardinalidad entre tablas:

```
usuarios (PK: id, email UNIQUE, rol, activo)
|
|-- [1:N] cursos (PK: id, FK: docente_id → usuarios.id, codigo_acceso UNIQUE)
|
```



```

|           |-- [1:N] temas (PK: id, FK: curso_id → cursos.id, orden)
|           |
|           |           |-- [1:N] materiales (PK: id, FK: tema_id → temas.id, tipo)
|           |           |
|           |           |-- [1:N] tareas (PK: id, FK: tema_id → temas.id, fecha_limite)
|           |           |
|           |           |-- [1:N] entregas (PK: id, FK: tarea_id, estudiante_id
|           |           |
|           |           |-- [1:1] calificaciones (PK: id, FK: en
trega_id)
|           |
|           |-- [1:N] inscripciones (PK: id, FK: estudiante_id, curso_id – UNIQUE)
|           |
|           |-- [1:N] anuncios (PK: id, FK: curso_id, autor_id → usuarios.id)
|           |
|-- [1:N] notificaciones (PK: id, FK: usuario_id → usuarios.id, tipo, leído)

```

4.3 Restricciones de Integridad

- **UNIQUE(email)** en usuarios — no pueden existir dos cuentas con el mismo correo.
- **UNIQUE(codigo_acceso)** en cursos — cada curso tiene un código irrepetible.
- **UNIQUE(estudiante_id, curso_id)** en inscripciones — un estudiante no puede inscribirse dos veces al mismo curso.
- **CASCADE DELETE** en cursos → temas → materiales/tareas → entregas/calificaciones — eliminar un curso borra todos sus contenidos en cadena.
- **CASCADE DELETE** en usuarios → inscripciones y notificaciones — eliminar un usuario borra sus registros dependientes.

5. Tecnologías Utilizadas

5.1 Backend

Tecnología	Descripción y uso en el proyecto
Python 3.11	Lenguaje principal del backend. Versión con soporte LTS y módulo zoneinfo nativo para manejo de zonas horarias sin dependencias externas.
Flask 3.x	Microframework web ligero y flexible. Se utilizó el patrón Application Factory con <code>create_app()</code> para permitir múltiples instancias configurables y facilitar las pruebas.
Flask-Login	Extensión para gestión de sesiones de usuario. Provee el decorador <code>@login_required</code> , la función <code>current_user</code> y el ciclo de login/logout seguro.
Flask-Migrate	Gestión de migraciones de base de datos basada en Alembic. Permite evolucionar el esquema sin pérdida de datos mediante comandos <code>flask db migrate / upgrade</code> .
SQLAlchemy (ORM)	Mapeador objeto-relacional que abstrae las operaciones de base de datos. Se utilizaron relaciones <code>lazy='dynamic'</code> , <code>back_populates</code> , <code>cascade</code> y <code>UniqueConstraint</code> .
Werkzeug	Biblioteca WSGI base de Flask. Se utilizó <code>werkzeug.security</code> para hashing de contraseñas (<code>generate_password_hash / check_password_hash</code> con PBKDF2-SHA256).

5.2 Base de Datos

Tecnología	Descripción y uso en el proyecto
SQLite	Base de datos relacional embebida utilizada en desarrollo. No requiere servidor separado; el archivo <code>.db</code> se gestiona directamente por SQLAlchemy. Para producción se recomienda migrar a PostgreSQL.
zoneinfo	Módulo de la biblioteca estándar de Python 3.9+ para manejo de zonas horarias. Se utilizó <code>ZoneInfo('America/La_Paz')</code> para garantizar que todas las fechas del sistema usen UTC-4 (Bolivia).

5.3 Frontend

Tecnología	Descripción y uso en el proyecto
Tailwind CSS 3	Framework CSS utilitario. Se compiló con la CLI de Tailwind generando el archivo <code>output.css</code> optimizado. Permite un diseño oscuro/claro sin CSS personalizado adicional.
Jinja2	Motor de plantillas integrado en Flask. Se utilizaron herencia de templates (<code>extends/block</code>), inclusión de componentes (<code>include</code>) y filtros para formateo de fechas y textos.
JavaScript (ES6)	Vanilla JS para interacciones del lado del cliente: filtrado de tablas en tiempo real, toggle del sidebar, cambio de tema oscuro/claro con persistencia en <code>localStorage</code> .
Chart.js	Librería de gráficos JavaScript utilizada en el dashboard para visualización de métricas estadísticas con gráficos de barras y donut.

5.4 Herramientas de Desarrollo

Tecnología	Descripción y uso en el proyecto
Git / GitHub	Control de versiones y repositorio remoto del proyecto. Se trabajó con commits descriptivos siguiendo convenciones de mensajes semánticos.
Linux Mint 22.2	Sistema operativo principal de desarrollo. Entorno configurado con Python, Node.js, Tailwind CLI y PostgreSQL para pruebas de despliegue.
VS Code	Editor de código principal con extensiones para Python (Pylance), Jinja2, Tailwind CSS IntelliSense y GitLens.
Render / Heroku	Plataformas de despliegue en la nube. El proyecto incluye Procfile y variables de entorno para despliegue en producción.

6. Uso de Herramientas de Inteligencia Artificial

6.1 Herramientas Utilizadas

Durante el desarrollo del proyecto se utilizó **Claude (Anthropic)**, específicamente el modelo **Claude Sonnet 4.6**, como asistente de inteligencia artificial principal. La interacción se realizó a través de la interfaz web de **claude.ai**.

6.2 Finalidad de Uso

- **Generación de código:** creación de templates HTML complejos con Jinja2 y Tailwind CSS, especialmente las vistas del panel de administración con sidebar colapsable, tablas con filtros dinámicos y modales de confirmación reutilizables.
- **Depuración y corrección de errores:** identificación y solución del bug de zona horaria (`datetime.utcnow()` vs `datetime.now(tz=Bolivia)`) que causaba que las tareas aparecieran vencidas inmediatamente después de crearse.
- **Arquitectura y diseño:** consultas sobre la mejor forma de estructurar el panel de administración, decisiones sobre separación de templates (`base_admin.html` vs `base.html`) y patrones para reutilización de componentes.
- **Consultas técnicas:** resolución de dudas sobre SQLAlchemy (cascade delete, relaciones lazy), Flask Blueprints, Flask-Login, y manejo de archivos estáticos con Tailwind CLI.
- **Documentación:** generación del presente informe académico en formato PDF con estructura profesional usando la librería ReportLab.

6.3 Descripción de las Consultas Realizadas

Área	Consulta realizada	Resultado obtenido
Panel Admin	¿Cómo separar el layout del administrador del la zona de entregas tardías con sidebar colapsable?	Creación de <code>base_admin.html</code> que extiende independientemente de <code>base.html</code> .
Zona horaria	Las tareas aparecen vencidas al crearlas, ¿cuál es la causa?	Identificación del bug: <code>datetime.utcnow()</code> vs hora local Bolivia. Solución con <code>datetime.now(tz=Bolivia)</code> .
Modales	¿Cómo reutilizar el componente modal.html existente?	Sustitución de las llamadas al <code>modal.html</code> por <code>{% include 'components/modal.html' %}</code> dentro de <code>base_admin.html</code> .
Entregas tardías	¿Cómo mostrar al docente qué estudiantes entregaron tarde?	Implementación del tiempo de entrega tardía con cálculo de <code>diff</code> en minutos/horas.
CRUD Admin	¿Qué operaciones debería tener el administrador de usuarios y cursos?	Definición del alcance de CRUD para usuarios y cursos, lectura+eliminación.

Tabla 4: Resumen de consultas realizadas a la IA durante el desarrollo

6.4 Reflexión sobre el Aporte de la IA al Proyecto

El uso de Claude como asistente de desarrollo representó una aceleración significativa en la fase de implementación de la interfaz de usuario y en la resolución de errores técnicos específicos. Sin embargo, es importante destacar que la IA operó como una herramienta de apoyo y no como sustituto del proceso de aprendizaje.

Las consultas más valiosas no fueron las de generación de código directa, sino las de **diagnóstico y razonamiento**: cuando las tareas aparecían como vencidas, la IA no solo corrigió el bug sino que explicó la causa raíz (diferencia entre `datetime.utcnow()` que devuelve UTC y `datetime.now()` que devuelve hora local, y cómo el `input type='datetime-local'` siempre opera con hora local del navegador). Esto convierte cada corrección en un aprendizaje concreto.

Por otro lado, la IA requirió guía constante: fue necesario proporcionar el contexto completo del proyecto (estructura de carpetas, modelos existentes, rutas actuales) en cada consulta para obtener respuestas útiles. Esto refuerza que la herramienta amplifica las capacidades del desarrollador, pero no puede reemplazar el entendimiento profundo de la arquitectura del sistema.

En conclusión, la integración de IA como asistente de desarrollo es una práctica válida y recomendable en proyectos académicos, siempre que se mantenga el criterio técnico del desarrollador para evaluar, adaptar y comprender el código generado antes de incorporarlo al proyecto.

7. Conclusiones y Aprendizajes

7.1 Conclusiones

El patrón Application Factory es fundamental para proyectos Flask escalables.

La organización del proyecto mediante `create_app()`, extensiones centralizadas y Blueprints independientes por módulo demostró ser la arquitectura correcta para un sistema de esta magnitud. Permite agregar nuevos módulos sin modificar código existente y facilita la configuración por entornos (desarrollo, pruebas, producción).

El manejo de zonas horarias es un aspecto crítico en sistemas con fechas límite.

El bug de zona horaria fue uno de los problemas más instructivos del proyecto. La diferencia entre `datetime.utcnow()` (UTC) y la hora enviada por el navegador (local) causaba inconsistencias silenciosas. La solución centralizada en `app/utils/timezone.py` con `zoneinfo` garantiza que toda la aplicación opera en la misma referencia temporal, patrón que debe aplicarse desde el inicio del diseño y no como corrección posterior.

La separación de layouts por rol mejora significativamente la experiencia de usuario.

Diseñar un `base_admin.html` completamente independiente del `base.html` de docentes/estudiantes fue una decisión arquitectónica acertada. El administrador opera en un contexto funcional diferente (gestión del sistema) y merece una interfaz orientada a esa tarea, no una adaptación del entorno pedagógico.

Tailwind CSS permite construir interfaces profesionales sin escribir CSS personalizado.

La utilización exclusiva de clases utilitarias de Tailwind CSS para todo el diseño, incluyendo el sidebar colapsable con transiciones CSS, el modo oscuro/claro con toggle persistente en `localStorage`, y las tarjetas de cursos con colores dinámicos, demostró que es posible lograr interfaces de alta calidad visual sin una sola línea de CSS propio.

La reutilización de componentes Jinja2 reduce errores y mantiene coherencia visual.

El componente `components/modal.html` utilizado en todas las acciones de confirmación del panel de administración es un ejemplo concreto de DRY (Don't Repeat Yourself) en templates. Cambiar el estilo del modal en un solo archivo impacta positivamente en toda la aplicación.

7.2 Aprendizajes Técnicos

- Implementación del patrón Application Factory en Flask con separación por Blueprints y extensiones centralizadas.
- Diseño de bases de datos relacionales con SQLAlchemy: relaciones bidireccionales, cascade delete, UniqueConstraint y lazy loading.

- Gestión de zonas horarias en Python con el módulo zoneinfo y la importancia de unificar la referencia temporal en toda la aplicación.
- Construcción de interfaces responsivas con Tailwind CSS, incluyendo sidebar colapsable con JavaScript y toggle de tema claro/oscuro.
- Implementación de autenticación con Flask-Login, hashing de contraseñas con Werkzeug y control de acceso basado en roles.
- Uso de Jinja2 avanzado: herencia de templates, inclusión de componentes con variables, filtros personalizados y operadores ternarios.
- Manejo de archivos subidos en Flask con werkzeug.utils.secure_filename, validación de extensiones y almacenamiento con nombres UUID.
- Integración responsable de herramientas de IA como asistente de desarrollo, manteniendo el criterio técnico propio para evaluar el código generado.

7.3 Trabajo Futuro

- Migración de SQLite a PostgreSQL para entornos de producción con mayor concurrencia.
- Implementación del buscador global en el navbar (buscar cursos, tareas y materiales simultáneamente).
- Notificaciones en tiempo real mediante WebSockets (Flask-SocketIO) para eliminar la necesidad de refrescar la página.
- Módulo de perfil de usuario con edición de datos personales y carga de avatar.
- Generación de reportes de calificaciones en PDF por curso con ReportLab.
- Tests automatizados con pytest y Flask-Testing para validar las rutas críticas del sistema.

El desarrollo del **Aula Virtual Académica** permitió aplicar de forma integral los conceptos de Tecnologías Emergentes en el contexto del desarrollo web moderno. El resultado es un sistema funcional, visualmente profesional y arquitectónicamente sólido que puede ser desplegado en producción y utilizado por instituciones educativas reales.